

## Prolog Program to Implement Monkey Banana Problem

### 1. Aim

To write and execute a Prolog program to solve the classic Monkey-Banana problem, in which a monkey must push a box under a banana, climb it, and grab the banana, using state-space search.

### 2. Objective

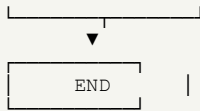
- To understand state-space representation in Prolog using a compound state term.
- To represent possible actions (walk, push, climb, grab) as state-transition rules.
- To implement goal-directed search that finds a sequence of actions reaching the goal.
- To retrieve whether the monkey can obtain the banana from a given starting state.

### 3. Algorithm

1. Start the program.
2. Represent the world state as `state(MonkeyPos, BoxPos, BananaPos, OnBox, HasBanana)`.
3. Define `move/3` facts describing each possible action and the resulting state: `grab`, `climb`, `push`, `walk`.
4. Define `canget/1`: a state is solved if the monkey already has the banana.
5. Recursive case: `canget/1` succeeds if some move leads to a state from which the banana can still be obtained.
6. Enter the initial state as a query to `canget/1`.
7. Prolog searches through the chain of moves using backtracking.
8. If a sequence of moves reaches the goal, display success; otherwise Prolog returns false.
9. Stop the program.

### 4. Flowchart





## 5. Code with Output

### Code:

```
% -----
% Monkey Banana Problem
% state(MonkeyPos, BoxPos, BananaPos, OnBox, HasBanana)
% -----

move(state(middle, atwindow, middle, onbox, hasnot),
    grab,
    state(middle, atwindow, middle, onbox, has)).

move(state(P, atwindow, P, onfloor, H),
    climb,
    state(P, atwindow, P, onbox, H)).

move(state(P1, atwindow, P1, onfloor, H),
    push(P1, P2),
    state(P2, atwindow, P2, onfloor, H)).

move(state(P1, B, Banana, onfloor, H),
    walk(P1, P2),
    state(P2, B, Banana, onfloor, H)).

% -----
% Goal test and search
% -----
canget(state(_, _, _, _, has)).
canget(State1) :-
    move(State1, _, State2),
    canget(State2).
```

### Output:

```
?- canget(state(atdoor, atwindow, middle, onfloor, hasnot)).
true.
```

## 6. Result

The Prolog program was successfully implemented to solve the Monkey-Banana problem, finding a valid sequence of actions (walk, push, climb, grab) that lets the monkey reach the banana.

## 7. Conclusion

Thus, the Monkey-Banana problem was successfully solved using Prolog. The experiment demonstrates state-space search, where states are represented as terms and Prolog's backtracking automatically explores the space of possible action sequences to reach the goal.